Assignment Report

SFWRENG 2AA4 (2026W)

Assignment 1

Author(s)

Krisha Patel	patek233@mcmaster.ca	patek233
Raadhikka Gupta	radhikar@mcmaster.ca	radhikar
Ranica Chawla	chawlr11@mcmaster.ca	chawlr11
Nitya Patel	paten131@mcmaster.ca	paten131

GitHub URL

`https://github.com/Nitya-NP/2AA4_Team5_Assignment1`

March 20, 2026

Original template credits: Samuel Zurowski and Charles Barone. Adapted for McMaster SFWRENG by Dr. Istvan David.

1 Executive summary

2 Requirements traceability

Table 1: Requirements Traceability Matrix

Req ID	Status	Classes	Design considerations
R3.1	Implemented	`BuildCityCommand.java`, `BuildRoadCommand.java`, `BuildSettlementCommand.java`, `Command.java`, `CommandManager.java`	Implement an undo/redo functionality
R3.2	Implemented	`AIAction.java`, `ConstraintChecker.java`, `ActionEvalutor.java`, `RuleBasedAI.java`, `ComputerPlayer.java`	Implement the Robber mechanism (see 4a in the rulebook) with the following simplifications. In case of rolling a 7, players may lose cards (if they have more than seven – see the rulebook), the Robber shall be placed on a random tile, and subsequently a qualifying player (i.e., one with a settlement or city adjacent to the robber's new tile) shall be randomly chosen to pass a random card to the player who rolled 7. • Earning a VP: 1.0. • Building something without earning a VP (i.e., building a road or buying a development card): 0.8. • Spending cards in a way that less than 5 cards remain in the agent's hand: 0.5.

R3.3	Implemented	`AIAction.java`, `ConstraintChecker.java`, `ActionEvalutor.java`, `RuleBasedAI.java`, `ComputerPlayer.java`	Implement the following constraints. If any of these apply in a situation, the agent should first resolve that constraint before focusing on value-added actions. • If there are more than 7 cards: the agent must spend them. • If there exist two road segments that are at most two units away, the agent should try to buy roads to connect the segments. • If other players have a longest road that is at most one road shorter than the agent's, the agent should buy a connected road segment. (For example, the agent has a longest road of 7, another has 6 → buy road..

3 Getting ready for change

The following UML diagram represents the structure of our game system. It illustrates the main classes and the relationships between them.

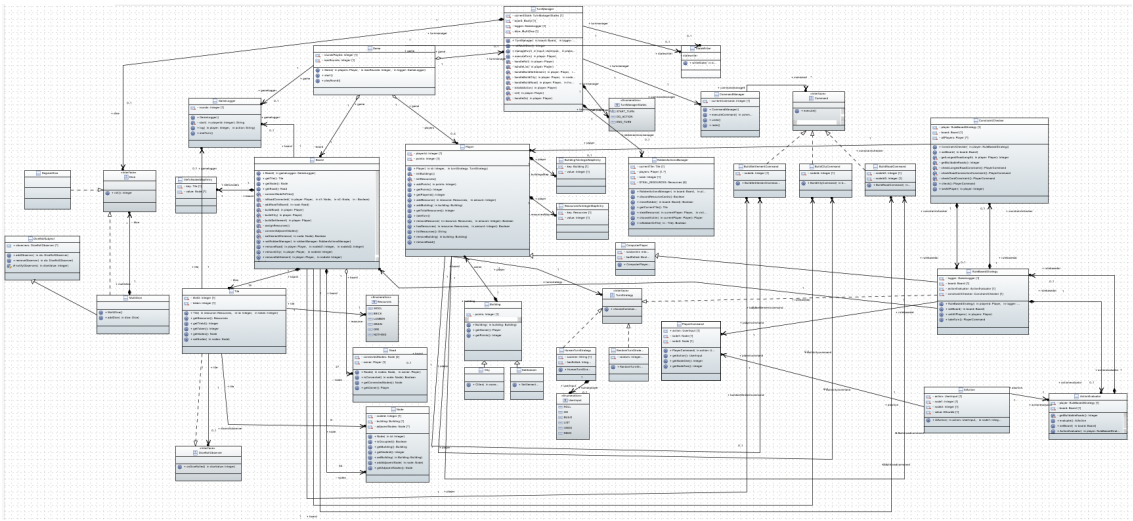

Figure 1: Design and domain modeling

4 Design pattern 1

Which design pattern did you choose and why?

We chose the Command Pattern because it lets us create each player action like building a road, city, or settlement as an object. This made it easy to support the undo and redo functionality, as each command can store the information needed to redo their actions.

Instead of calling methods like buildSettlement() in TurnManager, we made these actions into Command classes. It made the code simpler as they can be executed, undone, and redone. Furthermore, it improves scalability. If new actions need to be added in the future, they can be implemented as new command classes without changing existing code.

Explain the design pattern and elaborate on how it fosters cleaner design in this particular software.

The Command Pattern is a design pattern where each action in the program is represented as an object.

- **Command:** Command interface with execute(), undo(), and redo() functions
- **Command Classes:** BuildRoadCommand, BuildSettlementCommand, and BuildCityCommand
- **Invoker:** CommandManager - stores and manages the command history
- **Receiver:** HumanPlayer - calls the undo/redo action on the respective command

This helped make the design much cleaner because it separates what action is being done from how the system manages those actions over time. For example, building a road or upgrading a city is handled inside its own class, and the TurnManager class which uses the undo/redo system doesn't need to know how those actions work internally, but rather it just calls undo() or redo() on the command. This avoids having large, messy logic inside those classes, which also makes the code more organized and easier to extend. If we want to add a new type of action in the future, we can just create a new command class instead of modifying a lot of the existing code. This follows good design principles like keeping classes focused on one responsibility and making the system easier to maintain.

Overall, the Command Pattern works really well for undo/redo because it naturally allows us to store actions and reverse them, and makes the implementation more aligned with the behavior we wanted from the system.

How much did you need to think about the structure of the evolved program once you have figured out which design pattern to use?

Once we figured out that the Command pattern was the right fit, we did not need to completely rethink the whole structure of the evolved program, but we still had to think carefully about where each responsibility should go. We knew we needed a command interface, command classes for each action, and the turn manager to call all the undo/redo operations. So

in that sense, the design pattern made the structure easier to organize because it gave us a clear template to follow. Instead of putting all the logic inside one large controller or game class, we could break the system into smaller pieces with specific roles.

However, we still had to think about some details regarding the structure of the program. We still had to think carefully about which actions should be turned into commands, what data each command needed to store in order to undo correctly, and how the manager should track the current command position. This was due to the fact that some commands require remembering previous values or board states so that undo can accurately restore the system which required careful planning. For example, in BuildRoadCommand, we had to decide that the command should store the Board, Player, and the two node IDs. Without that information, the command would not be able to properly execute, undo, or redo the road-building action.

Overall, once we identified the pattern, the structure of the program became much clearer and allowed us not having to figure everything out from scratch, but rather focus on smaller details, such as what each command should store for their undo/redo functions. So the pattern made things simpler by removing a lot of complexity, but we still needed to plan the design carefully to properly execute the pattern.

How much change was needed in the design and in the implementation?

We made changes that were required in the design and implementation of the undo/redo functionality. Initially, player actions were handled in TurnManager. To apply Command Pattern, we had to refactor these action methods into separate command classes.

We also added a CommandManager class that would keep track of all executed commands and manage the undo/redo operations. This needed us to think about how the actions would be called during a player's turn.

In addition, we had to modify the Board and Player classes to help with undoing actions. We had to implement methods to remove buildings and roads and make sure that the player got those resources back.

Although these changes required restructuring the code, the core logic of the game was the same. The changes improved the organisation of the code and helped make it easy to maintain.

5 Design Pattern 2

Which design pattern did you choose and why?

We chose the Observer Pattern for handling dice roll notifications in the Catan game. In Catan, when dice are rolled, multiple tiles need to react by producing resources. This creates a one to many dependency: one MultiDice object and many Tile objects. The Observer

pattern perfectly captures this relationship where when one object changes state, all its dependents are automatically notified and updated. It decouples the dice from the tiles, the dice doesn't need to know which tiles exist or how they produce resources.

Explain the design pattern and elaborate on how it fosters cleaner design in this particular software.

The Observer Pattern defines a one to many dependency between objects so that when one object changes state, all its dependents are notified and updated.

- Subject: DiceRollSubject abstract class extends MultiDice
- Observer: DiceRollObserver interface implemented by Tile
- Notification: When MultiDice.roll() is called, it calls notifyObservers(diceValue)

Cleaner design:

- Loose Coupling: MultiDice doesn't need to know anything about Tile, it just knows it has observers. Tiles can be added or removed without modifying the dice class (but in this game we don't need to remove the tile). New observer types could be added later that require dice roll.
- Open/Closed Principle: We can add new observers without modifying the subject. The system is open for extension but closed for modification.
- Single Responsibility: MultiDice only handles rolling dice and notification. Tile only handles resource production logic. Board class only handles building logics. Each class had one reason to change.
- Simplified Communication: Without Observer, we would need to manually call each tile's method after every dice roll. The pattern automates this communication.
- Tiles register themselves with dice at runtime in Board.registerTileWithDice(). We could add or remove tiles if needed.

How much did you need to think about the structure of the evolved program once you have figured out which design pattern to use?

Moderate thought was needed:

- Initial Analysis:
  - We identified that dice rolls affect multiple tiles.
  - We realized that is a classical observer scenario.
  - We decided on the observer pattern after some discussion.
- Planning:
  - We determined what should be the subject (MultiDice), observer (Tile) and notification method (onDiceRolled(int diceValue).

  - In this case Board class was the best choice for registration as it has references to Tile, robber and other classes needed.

- Other:
  - We had to consider how the robber blocks production (Dice == 7).
  - Needed to pass dependencies to tiles (like robber, logger) and ensured tiles only produce when their token matches the dice value.

How much change was needed in the design and in the implementation?

Moderate changes were required:

- Added new classes: DiceRollSubject (abstract class), DiceRollObserver (interface)
- Modified existing classes:
  - MultiDice: added extending DiceRollSubject, and in roll method notifyObserver(sum) is called.
  - Tile: Added implements DiceRollObserver and implements onDiceRolled() , distributedResources(), distributeResourceToPlayer(), and shouldSkipTile().
  - Board: Added registerTileWithDice() method and removed all resource production methods that previously existed (Relocating these methods to Tile class).

6 Design pattern 3

Which design pattern did you choose and why?

We chose the Strategy Pattern to handle how different players decide their actions during a turn. In our original design, we had separate classes like HumanPlayer and ComputerPlayer, which meant the behavior for choosing actions was tied directly to the player type, which made the design less flexible and harder to extend.

By using the Strategy Pattern, we moved the decision-making logic into separate strategy classes, such as HumanTurnStrategy, RandomTurnStrategy, and RuleBasedStrategy. By using this design, each player now can use a strategy object that determines how they choose their next command. This allows us to easily switch or add new behaviors without changing the Player class.

We chose this pattern because player behavior is something that can vary a lot (human input, random input, rule-based AI), and the Strategy design pattern is designed exactly for that situation. It also improves the design by following principles like Single Responsibility (player stores data, strategy handles decisions) and Open/Closed (we can add new strategies without modifying existing code) principles.

Explain the design pattern and elaborate on how it fosters cleaner design in this particular software.

The strategy pattern lets us design the different ways players can take a turn, and lets us switch easily between them. We created a TurnStrategy interface, and different classes such as HumanTurnStrategy, RandomTurnStrategy, and RuleBasedStrategy to implement it.

This makes the design better because instead of putting all the logic in Player, we move it to different Strategy classes. It makes the code clean and easier to understand, since every class has one responsibility. It also makes the code scalable, as we can easily add a new strategy without changing existing code. We can just add a new class that implements TurnStrategy.

How much did you need to think about the structure of the evolved program once you have figured out which design pattern to use?

After we decided to use the Strategy design pattern, we knew that all-turn related logic would be moved to separate classes, and the Player class would use them instead of having the logic itself. After that, we had to organize the classes correctly. We came up with an interface TurnStrategy, and made sure that all the strategy classes had the chooseCommand method. We also had to update the Player class to have a reference to a strategy and call it in a turn.

We had to do some planning to decide what logic would be in each strategy, but the pattern helped us to make that decision. This made it easy to implement new features, like when implementing the machine AI requirement.

How much change was needed in the design and in the implementation?

The change required in the design was moderate, as we had to remove the inheritance-based structure (HumanPlayer and ComputerPlayer extends Player) and replace it where Player contains a TurnStrategy object instead. This required updating the UML diagram to introduce the strategy interface and its implementations, and modifying relationships between classes.

Inside of the Player class, we had to refactor how turns are handled. Instead of calling something like takeTurn() directly on different player subclasses, we introduced a method like chooseCommand() in the Player class that delegates choosing the command to the strategy object instead of being directly in the Player. This meant moving input handling and AI logic out of the player classes into their respective strategy classes.

Although we didn't implement this pattern in the code but rather only in the design, the changes would be fairly localized to player behavior and turn handling. Once the strategy structure is set up, the rest of the system (like TurnManager and Game) becomes simpler as it does not need to care if a player is human or a computer anymore to get the current action in their turn.

7 Roles and responsibilities

- **Raadhikka Gupta** - contributed and updated the UML diagram to add new design patterns, implementing R3.1 (undo/redo functionality) in the code, worked on implementing a strategy design pattern on the UML, and the report.
- **Ranica Chawla** - worked on updating UML for Command design pattern, implementing R3.1 (undo/redo functionality) in the code, updating the UML to include the Strategy design pattern, and the report
- **Nitya Patel**
- **Krisha Patel**